

assignment_1_ols_visualize

August 11, 2026

1 Assignment 1: Training models

1.1 OLS Regression with visual diagnostics

This notebook fits a simple ordinary least squares (OLS) regression model for advertising budget and sales, then uses several views to understand the relationship, fit quality, residual behavior, and prediction uncertainty.

The project environment provides the plotting stack through uv: NumPy, pandas, Matplotlib, Seaborn, and statsmodels.

1.2 Analysis map

1. Load and validate the ten observations.
2. Explore the data with tables and overview plots.
3. Calculate OLS coefficients manually, then cross-check them with statsmodels.
4. Visualize the fitted line, confidence band, prediction interval, and USD 12,000 forecast.
5. Inspect residuals, normality, leverage, correlation, and explained variation.
6. Summarize the assignment answers and modeling limitations.

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from IPython.display import display
from scipy import stats

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams.update(
    {
        "figure.dpi": 110,
        "axes.titleweight": "bold",
        "axes.labelweight": "bold",
        "figure.constrained_layout.use": True,
    }
)
pd.set_option("display.float_format", lambda value: f"{value:,.4f}")
```

1.3 Problem setup

- **Advertising Budget (X):** amount spent on advertising, measured in thousands of dollars.
- **Sales (Y):** units sold, measured in thousands of units.

The simple linear model is $\hat{y} = b_0 + b_1 * x$, where b_0 is the intercept and b_1 is the slope.

```
[2]: df = pd.DataFrame(  
    {  
        "month": np.arange(1, 11),  
        "advertising_budget": [3.0, 5.0, 2.0, 7.0, 8.0, 1.0, 4.0, 6.0, 9.0, 10.  
→0],  
        "sales": [6.0, 9.0, 4.0, 10.0, 12.0, 3.0, 7.0, 8.0, 13.0, 15.0],  
    }  
)  
  
# Validate the raw data before calculating any model quantities.  
assert df.shape == (10, 3)  
assert df["month"].is_unique  
assert not df.isna().any().any()  
assert (df[["advertising_budget", "sales"]] >= 0).all().all()  
  
display(df.style.format({"advertising_budget": "{:.1f}", "sales": "{:.1f}"))
```

<pandas.io.formats.style.Styler at 0x10f03af60>

1.4 Descriptive statistics

The summary gives scale and spread before modeling. This prevents the regression coefficients from being interpreted without knowing the data range.

```
[3]: summary = (  
    df[["advertising_budget", "sales"]]  
    .describe()  
    .T.rename(columns={"mean": "mean", "std": "std_dev"})[  
        ["count", "mean", "std_dev", "min", "25%", "50%", "75%", "max"]  
    ]  
)  
display(summary)  
print(f"Pearson correlation: {df['advertising_budget'].corr(df['sales']):.6f}")
```

	count	mean	std_dev	min	25%	50%	75%	\
advertising_budget	10.0000	5.5000	3.0277	1.0000	3.2500	5.5000	7.7500	
sales	10.0000	8.7000	3.8887	3.0000	6.2500	8.5000	11.5000	
		max						
advertising_budget	10.0000							
sales		15.0000						

Pearson correlation: 0.986189

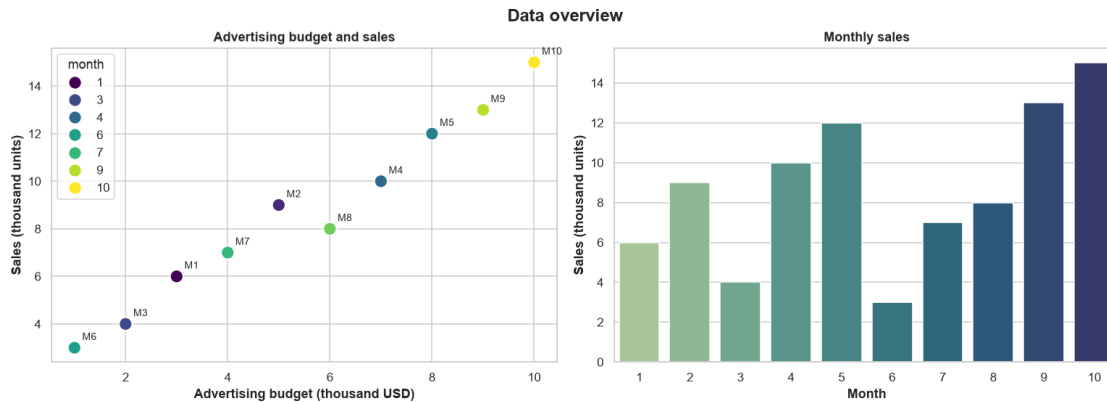
```

[4]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.scatterplot(
    data=df,
    x="advertising_budget",
    y="sales",
    hue="month",
    palette="viridis",
    s=140,
    edgecolor="white",
    linewidth=1.2,
    ax=axes[0],
)
for _, row in df.iterrows():
    axes[0].annotate(
        f"M{int(row['month'])}",
        (row["advertising_budget"], row["sales"]),
        xytext=(7, 7),
        textcoords="offset points",
        fontsize=9,
    )
axes[0].set_title("Advertising budget and sales")
axes[0].set_xlabel("Advertising budget (thousand USD)")
axes[0].set_ylabel("Sales (thousand units)")

sns.barplot(
    data=df,
    x="month",
    y="sales",
    hue="month",
    palette="crest",
    legend=False,
    ax=axes[1],
)
axes[1].set_title("Monthly sales")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Sales (thousand units)")
fig.suptitle("Data overview", fontsize=16, fontweight="bold")
display(fig)
plt.close(fig)

```



1.5 OLS calculation

For simple linear regression, calculate the coefficients with:

- $b_1 = S_{xy} / S_{xx}$
- $b_0 = y_{\text{mean}} - b_1 * x_{\text{mean}}$
- $S_{xy} = \sum((x_i - x_{\text{mean}}) * (y_i - y_{\text{mean}}))$
- $S_{xx} = \sum((x_i - x_{\text{mean}}) ** 2)$

The manual calculation is the primary implementation. `statsmodels` provides an independent cross-check and prediction intervals.

```
[5]: x = df["advertising_budget"].to_numpy(dtype=float)
y = df["sales"].to_numpy(dtype=float)
x_mean = x.mean()
y_mean = y.mean()
sxx = np.sum((x - x_mean) ** 2)
sxy = np.sum((x - x_mean) * (y - y_mean))
manual_slope = sxy / sxx
manual_intercept = y_mean - manual_slope * x_mean
manual_fitted = manual_intercept + manual_slope * x
manual_residuals = y - manual_fitted

print(f"x_mean = {x_mean:.6f}")
print(f"y_mean = {y_mean:.6f}")
print(f"Sxx = {sxx:.6f}")
print(f"Sxy = {sxy:.6f}")
print(f"Intercept (b0) = {manual_intercept:.6f}")
print(f"Slope (b1) = {manual_slope:.6f}")
print(f"Fitted line: y_hat = {manual_intercept:.4f} + {manual_slope:.4f}x")

assert np.isclose(manual_intercept, 1.7333333333)
assert np.isclose(manual_slope, 1.2666666667)
```

```

x_mean = 5.500000
y_mean = 8.700000
Sxx = 82.500000
Sxy = 104.500000
Intercept (b0) = 1.733333
Slope (b1) = 1.266667
Fitted line: y_hat = 1.7333 + 1.2667x

```

```

[6]: X = sm.add_constant(x, has_constant="add")
ols_results = sm.OLS(y, X).fit()

comparison = pd.DataFrame(
    {
        "manual OLS": [manual_intercept, manual_slope],
        "statsmodels OLS": ols_results.params,
    },
    index=["intercept", "slope"],
)
display(comparison)
print(f"Statsmodels R-squared: {ols_results.rsquared:.6f}")

assert np.allclose(ols_results.params, [manual_intercept, manual_slope])
assert np.isclose(ols_results.rsquared, 0.9725691893)

```

	manual OLS	statsmodels OLS
intercept	1.7333	1.7333
slope	1.2667	1.2667

Statsmodels R-squared: 0.972569

1.6 Prediction at USD 12,000

The predictor is measured in thousands of dollars, so USD 12,000 corresponds to $x = 12.0$. The model's mean response interval and individual prediction interval show different uncertainty:

- **Confidence interval:** uncertainty around the mean sales response at a budget value.
- **Prediction interval:** uncertainty for one future observation, so it is wider.

```

[7]: prediction_budget_usd = 12_000
prediction_budget_thousands = prediction_budget_usd / 1_000
prediction_design = sm.add_constant(
    np.array([prediction_budget_thousands]),
    has_constant="add",
)
prediction_frame = ols_results.get_prediction(prediction_design).summary_frame(
    alpha=0.05
)
predicted_sales_thousands = float(prediction_frame["mean"].iloc[0])

```

```

prediction_table = pd.DataFrame(
    {
        "budget_usd": [prediction_budget_usd],
        "budget_thousands": [prediction_budget_thousands],
        "predicted_sales_thousands": [predicted_sales_thousands],
        "mean_ci_lower": [float(prediction_frame["mean_ci_lower"].iloc[0])],
        "mean_ci_upper": [float(prediction_frame["mean_ci_upper"].iloc[0])],
        "prediction_interval_lower": [float(prediction_frame["obs_ci_lower"].
↪iloc[0])],
        "prediction_interval_upper": [float(prediction_frame["obs_ci_upper"].
↪iloc[0])],
    }
)
display(prediction_table)
print(f"Predicted sales: {predicted_sales_thousands:.4f} thousand units")
print(f"Approximately {predicted_sales_thousands * 1_000:,.0f} units")

assert np.isclose(predicted_sales_thousands, 16.9333333333)

```

	budget_usd	budget_thousands	predicted_sales_thousands	mean_ci_lower	\
0	12000	12.0000	16.9333	15.7008	

	mean_ci_upper	prediction_interval_lower	prediction_interval_upper
0	18.1658	14.9332	18.9335

Predicted sales: 16.9333 thousand units
Approximately 16,933 units

```

[8]: x_grid = np.linspace(x.min() - 0.5, 12.75, 250)
grid_design = sm.add_constant(x_grid, has_constant="add")
grid_frame = ols_results.get_prediction(grid_design).summary_frame(alpha=0.05)

fig, ax = plt.subplots(figsize=(11, 6.5))
ax.axvspan(
    x.min(),
    x.max(),
    color="#2563eb",
    alpha=0.07,
    label="Observed budget range",
)
ax.fill_between(
    x_grid,
    grid_frame["obs_ci_lower"].to_numpy(),
    grid_frame["obs_ci_upper"].to_numpy(),
    color="#f59e0b",
    alpha=0.14,
    label="95% prediction interval",
)

```

```

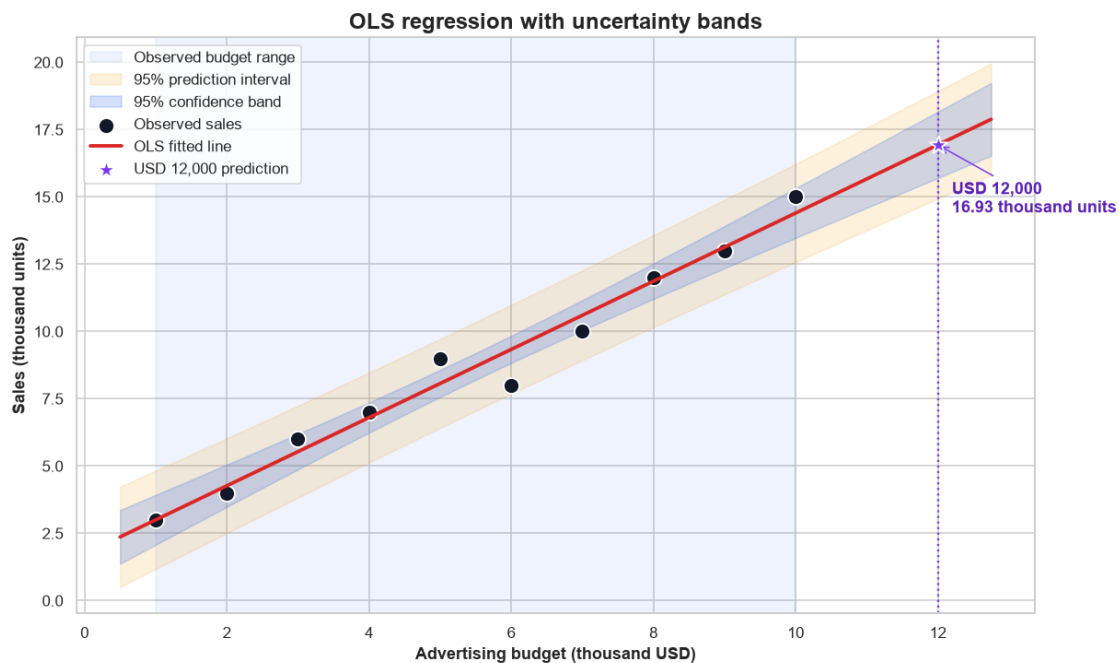
ax.fill_between(
    x_grid,
    grid_frame["mean_ci_lower"].to_numpy(),
    grid_frame["mean_ci_upper"].to_numpy(),
    color="#2563eb",
    alpha=0.20,
    label="95% confidence band",
)
sns.scatterplot(
    data=df,
    x="advertising_budget",
    y="sales",
    color="#111827",
    s=125,
    edgecolor="white",
    linewidth=1.2,
    label="Observed sales",
    ax=ax,
)
ax.plot(
    x_grid,
    grid_frame["mean"].to_numpy(),
    color="#dc2626",
    linewidth=2.5,
    label="OLS fitted line",
)
ax.scatter(
    [prediction_budget_thousands],
    [predicted_sales_thousands],
    color="#7c3aed",
    s=180,
    marker="*",
    edgecolor="white",
    linewidth=1.2,
    zorder=5,
    label="USD 12,000 prediction",
)
ax.axvline(prediction_budget_thousands, color="#7c3aed", linestyle=":",
↪ linewidth=1.5)
ax.annotate(
    f"USD 12,000\n{predicted_sales_thousands:.2f} thousand units",
    xy=(prediction_budget_thousands, predicted_sales_thousands),
    xytext=(10, -48),
    textcoords="offset points",
    arrowprops={"arrowstyle": "->", "color": "#7c3aed"},
    color="#5b21b6",
    fontweight="bold",

```

```

)
ax.set_title("OLS regression with uncertainty bands", fontsize=16)
ax.set_xlabel("Advertising budget (thousand USD)")
ax.set_ylabel("Sales (thousand units)")
ax.legend(loc="upper left", frameon=True)
display(fig)
plt.close(fig)

```



1.6.1 Coefficient interpretation

- **Slope:** each additional thousand dollars of advertising is associated with approximately 1.2667 thousand additional units sold.
- **Intercept:** the fitted model predicts approximately 1.7333 thousand units at zero advertising budget. Zero is outside the observed range, so this is an extrapolated baseline.
- **Prediction:** USD 12,000 gives approximately 16.9333 thousand units, or about 16,933 units. Since 12.0 is above the observed maximum of 10.0, this forecast is extrapolation.

```

[9]: influence = ols_results.get_influence()
diagnostics = df.assign(
    fitted=ols_results.fittedvalues,
    residual=ols_results.resid,
    studentized_residual=influence.resid_studentized_internal,
    leverage=influence.hat_matrix_diag,
    cooks_distance=influence.cooks_distance[0],
)

```



```

display(
    diagnostics[
        [
            "month",
            "sales",
            "fitted",
            "residual",
            "studentized_residual",
            "leverage",
            "cooks_distance",
        ]
    ].style.format(
        {
            "sales": "{:.3f}",
            "fitted": "{:.3f}",
            "residual": "{:.3f}",
            "studentized_residual": "{:.3f}",
            "leverage": "{:.3f}",
            "cooks_distance": "{:.3f}",
        }
    )
)
print(f"Residual mean: {diagnostics['residual'].mean():.6f}")
print(f"Residual standard deviation: {diagnostics['residual'].std(ddof=1):.6f}")

```

<pandas.io.formats.style.Styler at 0x10f50c590>

Residual mean: -0.000000

Residual standard deviation: 0.644061

```

[10]: fig, axes = plt.subplots(2, 2, figsize=(13, 10))

sns.scatterplot(
    data=diagnostics,
    x="fitted",
    y="residual",
    hue="month",
    palette="viridis",
    s=120,
    legend=False,
    ax=axes[0, 0],
)
axes[0, 0].axhline(0, color="#dc2626", linestyle="--", linewidth=1.5)
axes[0, 0].set_title("Residuals vs fitted values")
axes[0, 0].set_xlabel("Fitted sales (thousand units)")
axes[0, 0].set_ylabel("Residual")

```

```

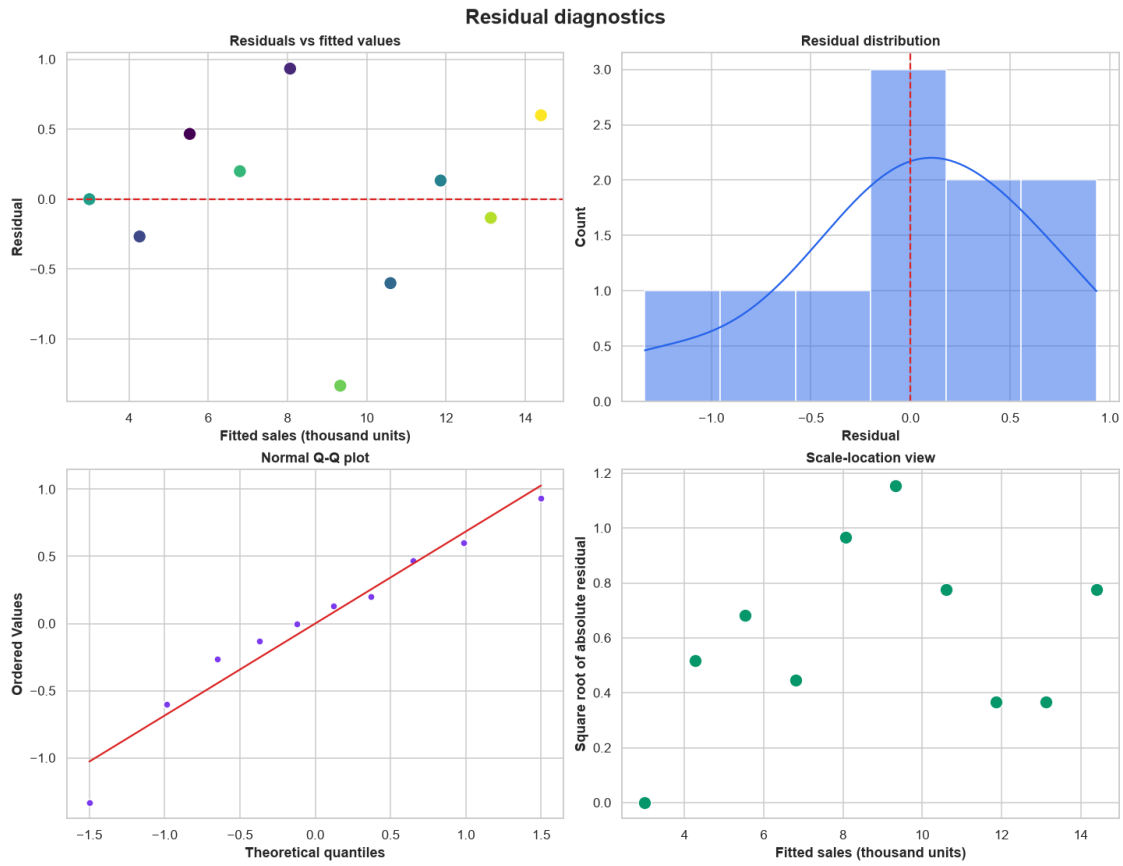
sns.histplot(
    diagnostics["residual"],
    bins=6,
    kde=True,
    color="#2563eb",
    edgecolor="white",
    ax=axes[0, 1],
)
axes[0, 1].axvline(0, color="#dc2626", linestyle="--", linewidth=1.5)
axes[0, 1].set_title("Residual distribution")
axes[0, 1].set_xlabel("Residual")

stats.probplot(diagnostics["residual"], dist="norm", plot=axes[1, 0])
axes[1, 0].set_title("Normal Q-Q plot")
axes[1, 0].get_lines()[0].set_markerfacecolor("#7c3aed")
axes[1, 0].get_lines()[0].set_markeredgecolor("white")
axes[1, 0].get_lines()[1].set_color("#dc2626")

diagnostics["sqrt_abs_residual"] = np.sqrt(np.abs(diagnostics["residual"]))
sns.scatterplot(
    data=diagnostics,
    x="fitted",
    y="sqrt_abs_residual",
    color="#059669",
    s=120,
    ax=axes[1, 1],
)
axes[1, 1].set_title("Scale-location view")
axes[1, 1].set_xlabel("Fitted sales (thousand units)")
axes[1, 1].set_ylabel("Square root of absolute residual")

fig.suptitle("Residual diagnostics", fontsize=17, fontweight="bold")
display(fig)
plt.close(fig)

```



1.6.2 Reading the residual plots

The residuals should be centered around zero without a curved pattern or a widening spread. With only ten observations, these diagnostics are descriptive rather than decisive. The Q-Q plot is a small-sample check for normality, not proof that the residuals are normally distributed.

```
[11]: fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

sns.scatterplot(
    data=diagnostics,
    x="sales",
    y="fitted",
    hue="month",
    palette="viridis",
    s=140,
    edgecolor="white",
    linewidth=1.2,
    ax=axes[0],
)
fit_limits = [
```

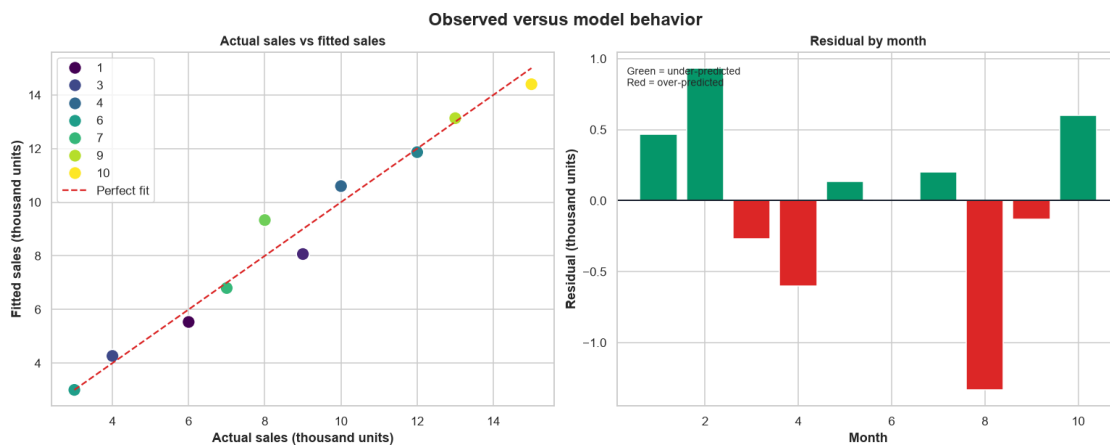
```

min(diagnostics["sales"].min(), diagnostics["fitted"].min()),
max(diagnostics["sales"].max(), diagnostics["fitted"].max()),
]
axes[0].plot(
    fit_limits, fit_limits, color="#dc2626", linestyle="--", label="Perfect fit"
)
axes[0].set_title("Actual sales vs fitted sales")
axes[0].set_xlabel("Actual sales (thousand units)")
axes[0].set_ylabel("Fitted sales (thousand units)")
axes[0].legend()

bar_colors = np.where(diagnostics["residual"] >= 0, "#059669", "#dc2626")
axes[1].bar(
    diagnostics["month"], diagnostics["residual"], color=bar_colors,
    edgecolor="white"
)
axes[1].axhline(0, color="#111827", linewidth=1.2)
axes[1].set_title("Residual by month")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Residual (thousand units)")
axes[1].text(
    0.02,
    0.96,
    "Green = under-predicted\nRed = over-predicted",
    transform=axes[1].transAxes,
    va="top",
    fontsize=9,
)

fig.suptitle("Observed versus model behavior", fontsize=16, fontweight="bold")
display(fig)
plt.close(fig)

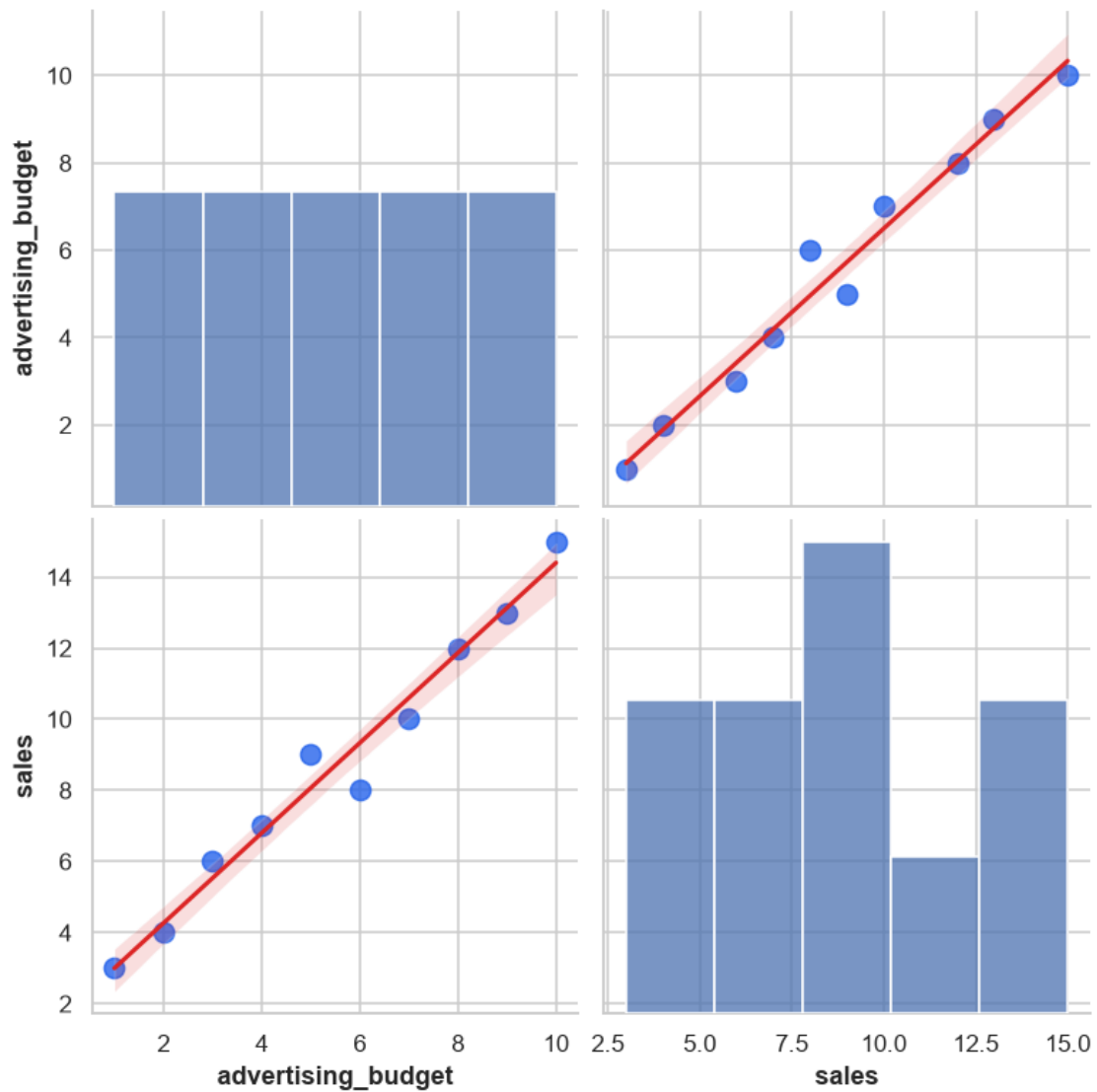
```



```
[12]: pair_grid = sns.pairplot(
    df[["advertising_budget", "sales"]],
    kind="reg",
    diag_kind="hist",
    height=3.6,
    plot_kws={
        "scatter_kws": {"s": 85, "alpha": 0.8, "color": "#2563eb"},
        "line_kws": {"color": "#dc2626", "linewidth": 2},
    },
)
pair_grid.fig.suptitle(
    "Pairwise distributions and linear relationship", y=1.02, fontsize=16
)
display(pair_grid.fig)
plt.close(pair_grid.fig)
```

```
/Users/beam/Workspace/Course/my-cpe-
lab/Y3/cpe342/lab01/.venv/lib/python3.12/site-packages/seaborn/axisgrid.py:123:
UserWarning: The figure layout has changed to tight
    self._figure.tight_layout(*args, **kwargs)
```

Pairwise distributions and linear relationship

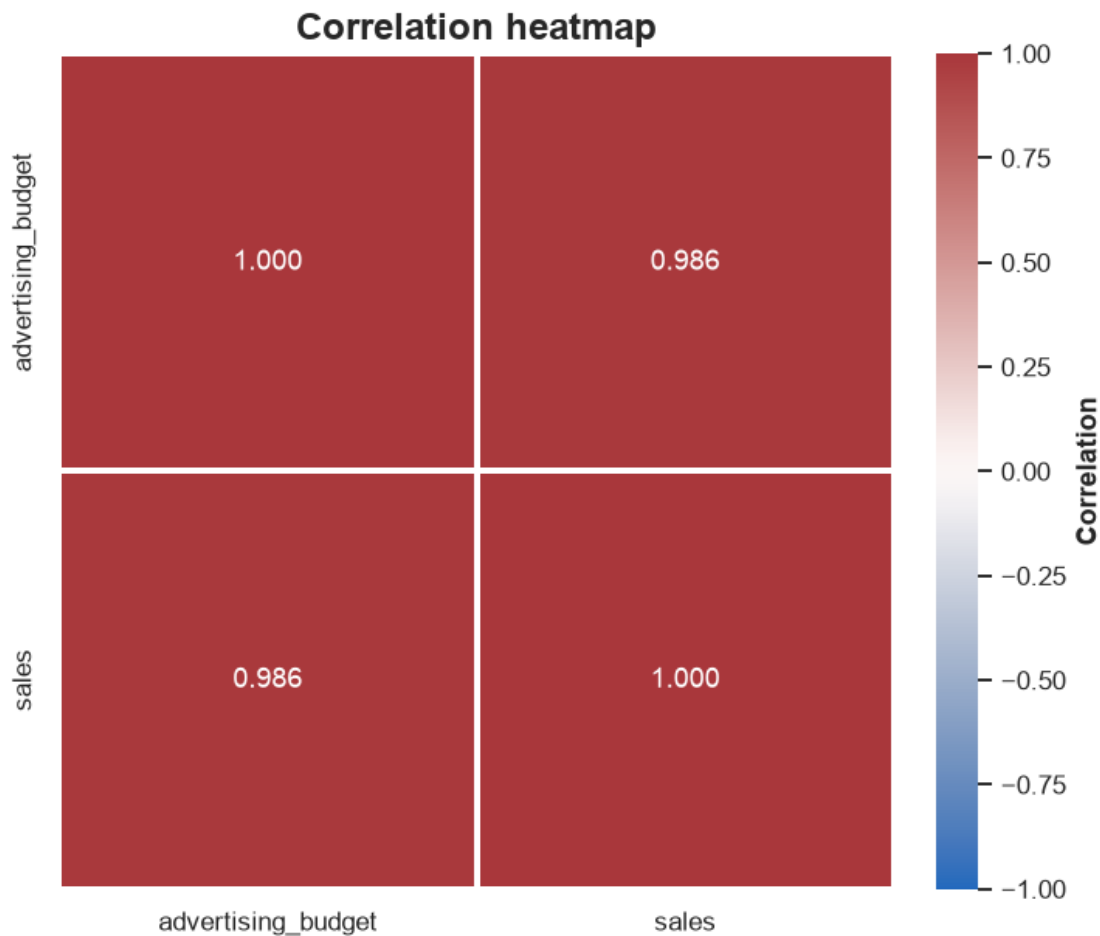


```
[13]: correlation = df[["advertising_budget", "sales"]].corr()
fig, ax = plt.subplots(figsize=(6.5, 5.5))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".3f",
    cmap="vlag",
    vmin=-1,
    vmax=1,
    square=True,
    linewidths=1.5,
```

```

cbar_kws={"label": "Correlation"},
ax=ax,
)
ax.set_title("Correlation heatmap", fontsize=16)
display(fig)
plt.close(fig)

```



1.7 Leverage and influence

Leverage identifies observations with unusual predictor values. Cook's distance combines leverage and residual size to flag observations that could strongly affect the fitted line. These are diagnostic signals, not automatic reasons to remove data.

```

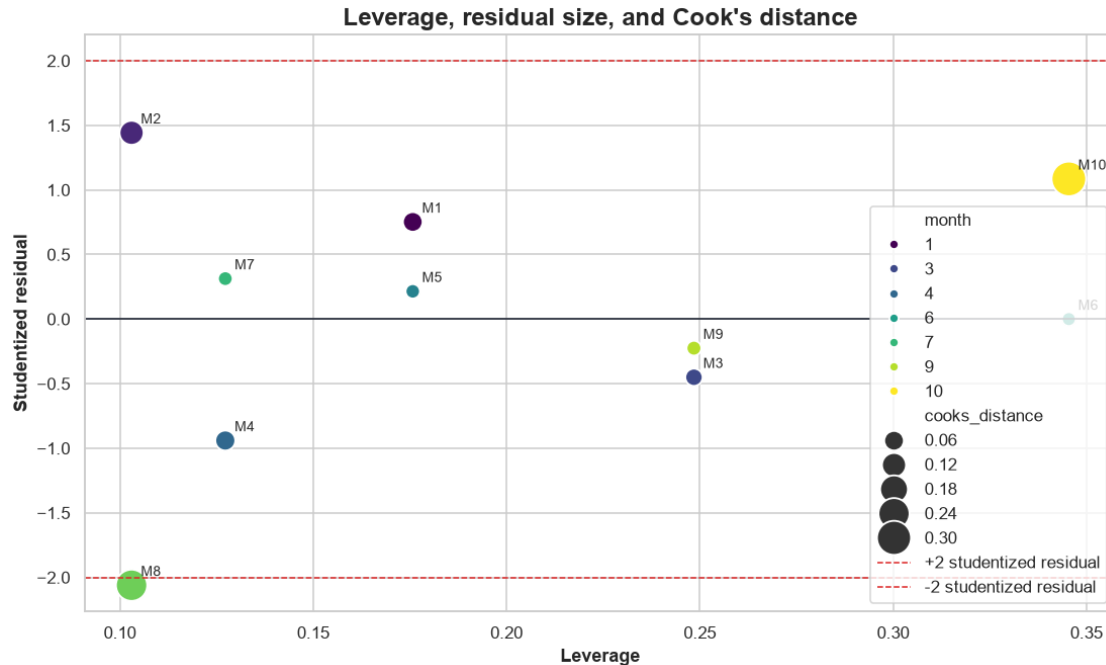
[14]: fig, ax = plt.subplots(figsize=(10, 6))
sns.scatterplot(
    data=diagnostics,
    x="leverage",
    y="studentized_residual",

```

```

    hue="month",
    size="cooks_distance",
    sizes=(80, 500),
    palette="viridis",
    edgecolor="white",
    linewidth=1.2,
    ax=ax,
)
for _, row in diagnostics.iterrows():
    ax.annotate(
        f"M{int(row['month'])}",
        (row["leverage"], row["studentized_residual"]),
        xytext=(6, 6),
        textcoords="offset points",
        fontsize=9,
    )
ax.axhline(0, color="#111827", linewidth=1)
ax.axhline(
    2, color="#dc2626", linestyle="--", linewidth=1, label="+2 studentized_↵
↵residual"
)
ax.axhline(
    -2, color="#dc2626", linestyle="--", linewidth=1, label="-2 studentized_↵
↵residual"
)
ax.set_title("Leverage, residual size, and Cook's distance", fontsize=16)
ax.set_xlabel("Leverage")
ax.set_ylabel("Studentized residual")
ax.legend(loc="best", frameon=True)
display(fig)
plt.close(fig)

```

1.8 R-squared and final evaluation

Calculate the coefficient of determination from the residual and total sums of squares:

- $SSE = \sum (y_i - \hat{y}_i)^2$
- $SST = \sum (y_i - \bar{y})^2$
- $R^2 = 1 - SSE / SST$

```
[15]: sse = np.sum((y - ols_results.fittedvalues) ** 2)
sst = np.sum((y - y_mean) ** 2)
r_squared = 1 - sse / sst

metrics = pd.DataFrame(
    {
        "metric": ["SSE", "SST", "R-squared", "Explained variation"],
        "value": [sse, sst, r_squared, r_squared * 100],
        "units": ["(thousand units)^2", "(thousand units)^2", "proportion",
        ↪ "percent"],
    }
)
display(metrics)
print(f"R-squared = {r_squared:.6f}")
print(f"Approximately {r_squared * 100:.2f}% of sample sales variation is
↪ explained.")

assert np.isclose(sse, 3.7333333333)
```

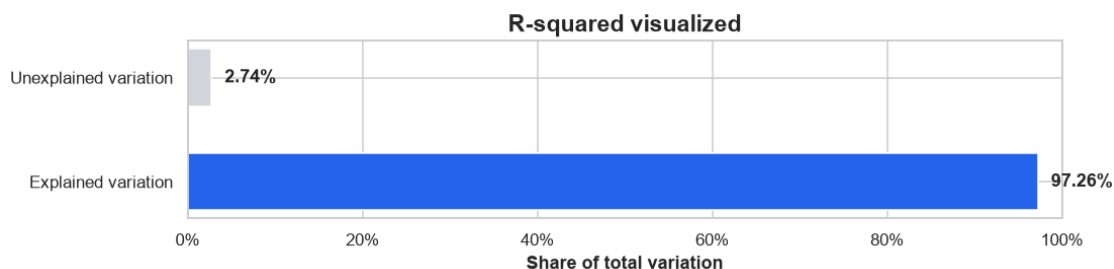
```
assert np.isclose(sst, 136.1)
assert np.isclose(r_squared, 0.9725691893)
```

	metric	value	units
0	SSE	3.7333	(thousand units) ²
1	SST	136.1000	(thousand units) ²
2	R-squared	0.9726	proportion
3	Explained variation	97.2569	percent

R-squared = 0.972569

Approximately 97.26% of sample sales variation is explained.

```
[16]: fig, ax = plt.subplots(figsize=(10, 2.4))
ax.barh(
    ["Explained variation", "Unexplained variation"],
    [r_squared, 1 - r_squared],
    color=["#2563eb", "#d1d5db"],
    edgecolor="white",
    height=0.55,
)
ax.set_xlim(0, 1)
ax.set_xlabel("Share of total variation")
ax.set_title("R-squared visualized", fontsize=15)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda value, _: f"{value:.0%}"))
for index, value in enumerate([r_squared, 1 - r_squared]):
    ax.text(value + 0.015, index, f"{value:.2%}", va="center",
            fontweight="bold")
display(fig)
plt.close(fig)
```



1.9 Final answers

- **Fitted equation:** $\text{Sales}_{\text{hat}} = 1.7333 + 1.2667 * \text{Advertising Budget}$.
- **Intercept:** 1.7333 thousand units sold at a zero advertising budget. Zero is outside the observed range.
- **Slope:** 1.2667 thousand additional units sold per additional thousand dollars of advertising budget. This is an association in the sample, not proof of causation.

- **Prediction at USD 12,000:** 16.9333 thousand units, approximately 16,933 units. This is extrapolation beyond the observed maximum budget of USD 10,000.
- **R-squared:** 0.972569, so approximately 97.26% of the observed sales variation is explained by the fitted linear relationship.

A high R-squared does not establish causation, guarantee future accuracy, or replace residual and influence diagnostics.